

Transmitting And Receiving Packets Using the STAR-System C++ API

STAR-Dundee's software stack, STAR-System, provides a consistent programming interface for accessing STAR-Dundee router and interface devices, including the STAR-Dundee SpaceWire PCI Mk2, PXI and PCIe devices and the STAR-Dundee SpaceWire Brick Mk3 and Router Mk2S devices. The C++ API provided with STAR-System provides classes for retrieving the list of devices connected to a PC, opening channels to access those devices, and then transmitting and receiving traffic over those channels.

This application note describes some of the C++ classes which can be used to transmit and receive packets, and includes some example code to demonstrate opening a channel to a device, transmitting a packet on that channel, receiving a packet on the channel, and then closing the channel.

A separate application note, "**Implementing A Simple RMAP Initiator Using The STAR-System C++ API**", expands upon this application note to demonstrate how the code presented here can be combined with the C++ API's RMAP Packet Library to implement a simple RMAP initiator. Further information on the APIs and the functions they provide can be found in the STAR-System C++ documentation.

SpaceWire Devices

The list of SpaceWire devices connected to the current PC can be obtained by declaring an instance of a `DeviceList` object (which can be optionally set to automatically populate itself with Device information) or by calling the `GetDeviceList()` method on the main `STARSystem` class. The `DeviceList` will be populated with a list of `Device` objects which can then be used to obtain information for each specific device present, such as the device name, its type and firmware version. In the example below a `DeviceList` object is declared and its `Populate()` method is called to populate the list with the list of devices connected to the current PC. The `Populate()` method returns the number of devices found as a `U32` variable. `U32`s are used in STAR-System to represent 32-bit unsigned integers. There are corresponding types provided for 8-bit and 16-bit unsigned integers (`U8` and `U16`). The `Populate()` method can be invoked numerous times if required to refresh the contents of the list with the latest information of any devices present.

```
DeviceList deviceList;
U32 deviceCount, i;

// populate the device list with any devices which are connected to the
// current PC, obtaining the number of devices found
deviceCount = deviceList.Populate();

// for each device found
for (i = 0; i < deviceCount; i++)
{
    // \todo do something with the device
}

// [optional] empty the device list when no longer required
// \note this step is not required since the contents of the device list will
// automatically be emptied when the DeviceList object goes out of scope
// or is deleted
deviceList.Empty();
```

The `Device` objects contained in the `DeviceList` can be iterated over in order to retrieve their properties. For example, a string representation of the type of a device, e.g. "SpaceWire PCI Mk2", can be obtained by calling the `GetDeviceTypeAsString()` method on the `Device` object. The `Device` object provides constants for expected string lengths such as `STR_DEVICE_TYPE_LEN`. See example below.

```
char deviceType[Device::STR_DEVICE_TYPE_LEN];

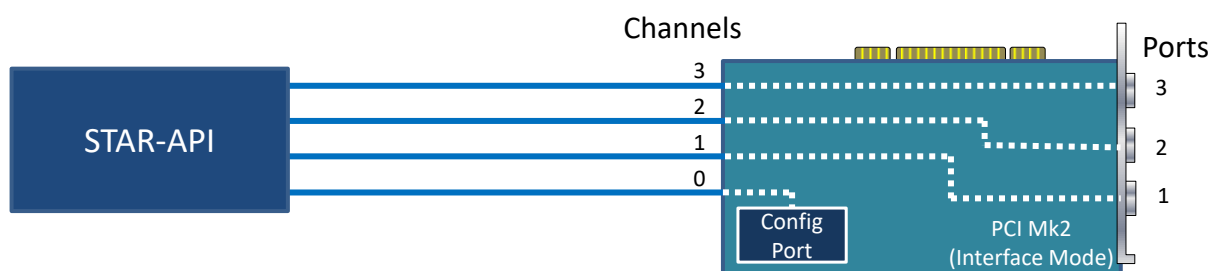
// for each device in the list
for (i = 0; i < deviceCount; i++)
{
    // get the device type as a string
    deviceList[i]->GetDeviceTypeAsString(deviceType);

    // display the device type
    cout << "Device type = " << deviceType << "." << endl;
}
```

Device Channels

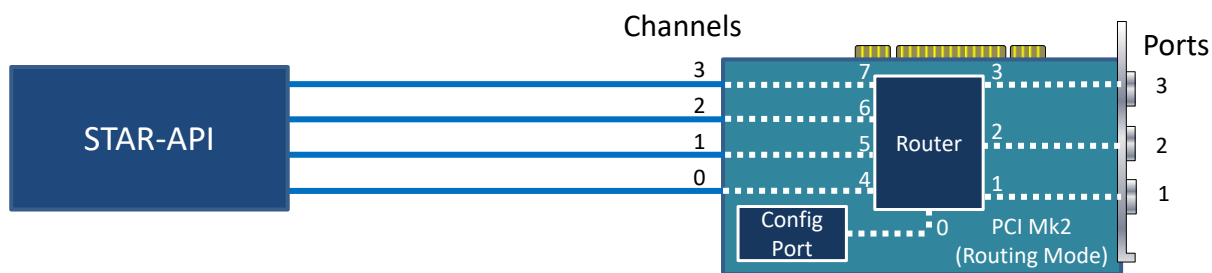
In STAR-System, all packets to be transmitted to a device and all packets received from a device are transferred between the device and the API over a previously opened channel. Each device has a certain number of channels which can be opened. On an interface device, or a device operating in interface mode, these channels are likely to attach directly to the links of the device. On a routing device, or a device operating in routing mode, these channels will attach to the external ports of the device's router.

The channels provided by a device can be obtained by calling the `GetDeviceChannels()` method on the `Device` object. This returns a bit mask indicating which of the channels exist for this device. The PCI Mk2, cPCI Mk2 and PCIe devices provide channels 0, 1, 2 and 3. When in interface mode, the default mode for these devices, channels 1, 2 and 3 correspond to links 1, 2 and 3, and any packets transmitted over these channels will be transmitted over the corresponding link. In this mode, channel 0 provides a channel for accessing the configuration port of the device, which is used by the Device Configuration APIs to configure the device. It is strongly recommended that channel 0 is not used for any other purpose, as traffic transmitted to this port will be passed directly to the configuration port. The following figure describes the relationship between channels and ports in interface mode on these devices.



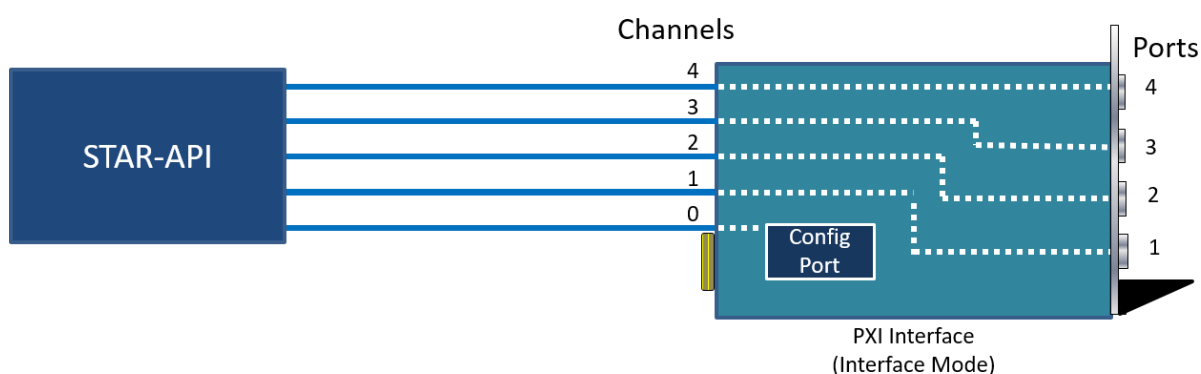
Relationship between channels and ports on a PCI device in interface mode

When these devices are in routing mode, channels 0, 1, 2 and 3 correspond to the four external ports on the device's router, ports 4, 5, 6 and 7. Channel 0 will still be used by the Device Configuration APIs to configure the device, so it's recommended that external port 4 is not used for any other purpose if the Configuration APIs are to be used. The figure below shows how traffic on each of the channels is routed to the ports when these devices are in routing mode.

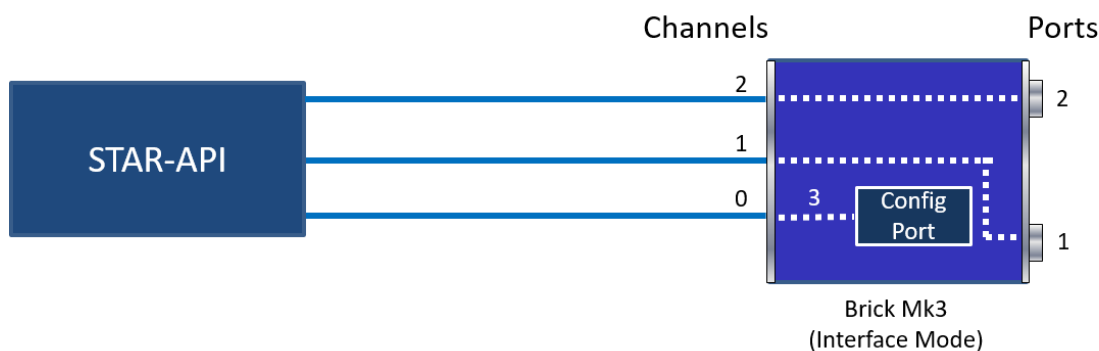


Relationship between channels and ports on a PCI device in routing mode

The PXI Interface and Brick Mk3 share a similar channel arrangement to the PCI devices, only differing in the number of ports on the device. The PXI Interface has 4 ports and channels 0, 1, 2, 3 and 4 whilst the Brick Mk3 has 2 ports and channels 0, 1 and 2. The following figures describe the relationship between channels and ports in interface mode on these devices.

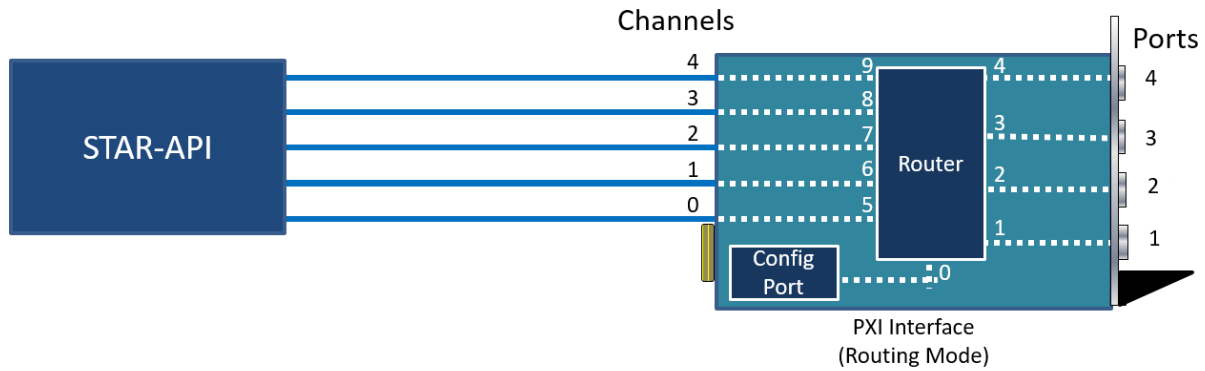


Relationship between channels and ports on a PXI Interface device in interface mode

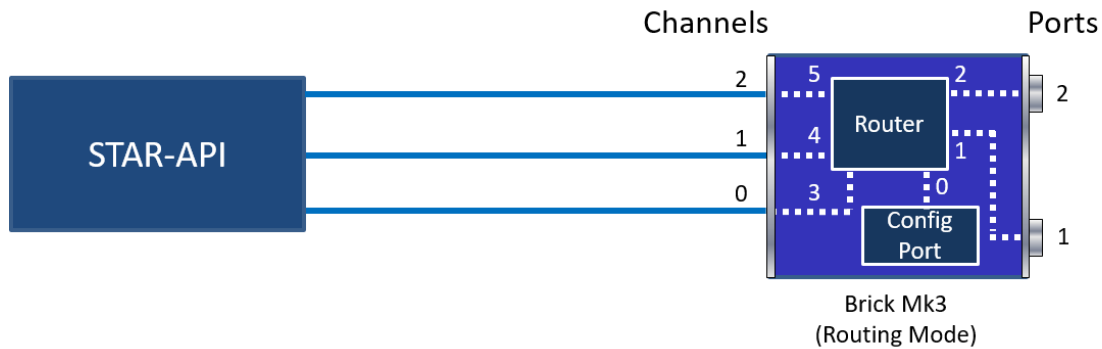


Relationship between channels and ports on a Brick Mk3 device in interface mode

The figures below show how traffic on each of the channels is routed to the ports when the PXI Interface and Brick Mk3 devices are in routing mode.

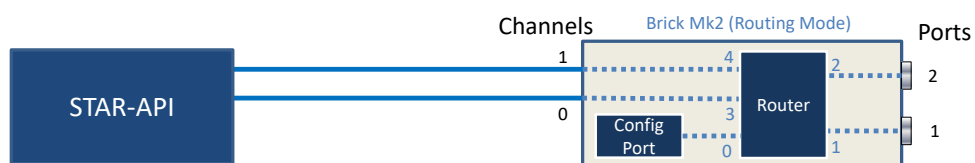


Relationship between channels and ports on a PXI Interface device in routing mode



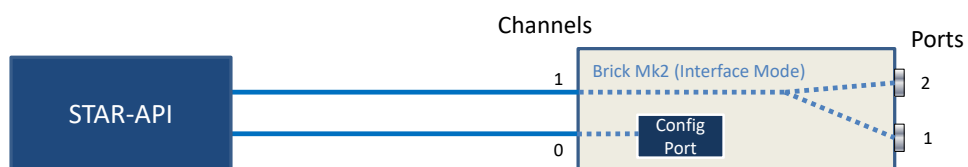
Relationship between channels and ports on a Brick Mk3 device in routing mode

The configuration of channels is a little different with the Brick Mk2 and Router Mk2S. Both devices have only two channels, channel 0 and channel 1. As with the other devices, channel 0 will be used by the Device Configuration APIs, leaving channel 1 for transmitting and receiving packets. This means that when in interface mode (the default for the Brick Mk2), packets to/from all links must cross the same channel. As a result, when in interface mode, these devices require transmitted packets to include an address byte (e.g. the port number to route the packet out of) at the front of the packet. When transmitting packets, the device is effectively operating as a router. The figure below shows the relationship between channels and ports when a Brick Mk2 is in routing mode, and also when transmitting packets in interface mode.



Relationship between channels and ports on a Brick Mk2 in routing mode

Packets received on all ports are routed up the same channel when in the default interface mode, which means the device does not operate as a true interface device in this mode, as traffic from one port can block traffic from another port. The figure below describes the relationship between channels and ports in interface mode on the Brick Mk2 when receiving.

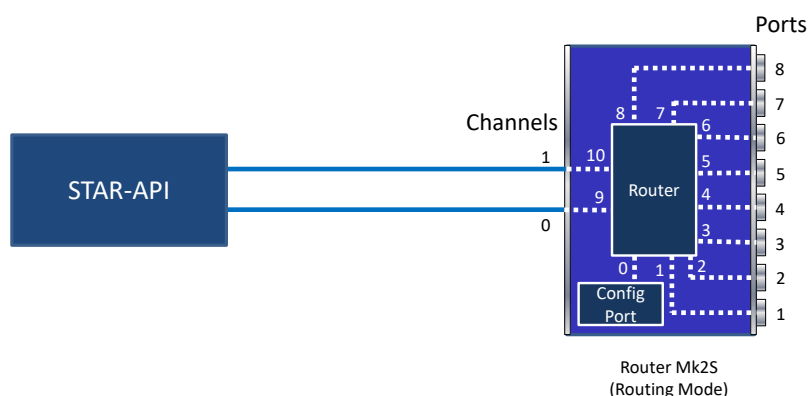


Relationship between channels and ports on a Brick Mk2 in interface mode when receiving

Although this would suggest that the Brick Mk2 cannot be used as a true interface device, all of the STAR-System interface and router devices support a mechanism called port routing. Port routing can be configured using the Device Configuration APIs, or using the Device Configuration application. Port routing allows a device to be configured so that all packets received on a port are automatically routed to another port. As a result, you can configure a Brick Mk2 so that packets received on port 1 are routed to port 3 (channel 0), and packets received on port 2 are routed to port 4 (channel 1). Note that if you're receiving packets on channel 0, you will not be able to use any of the Device Configuration API functions.

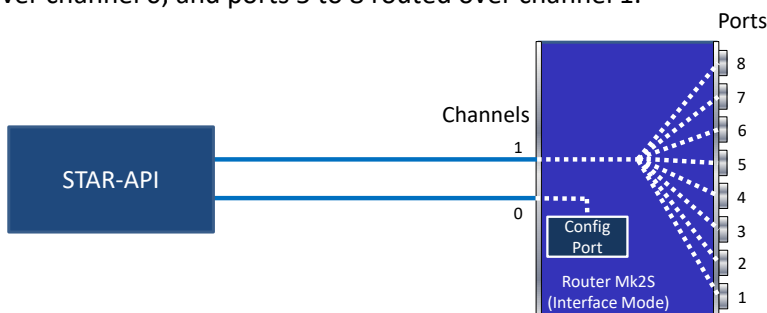
If you do not wish to add an address byte to the front of packets to be transmitted, you can also use port routing on the external ports. You could setup port routing so that packets received on port 4 (channel 1) are routed to port 2, and packets received on port 3 (channel 0) are routed to port 1. Note that if you do set this second port routing arrangement, you will not be able to access the configuration port, and as a result, use the Device Configuration APIs. This means you will not be able to change the port routing settings until the device is reset.

The Router Mk2S is very similar to the Brick Mk2, although it obviously has more ports. By default the device is in routing mode, as shown in the following figure. External ports 9 and 10 on the router correspond to channels 0 and 1.



Relationship between channels and ports on a Router Mk2S in routing mode

When in interface mode, packets received from all ports are routed up channel 1, with the Device Configuration APIs using channel 0, as shown in the figure below. Again, port routing can be used to configure two ports to act as real interfaces, or to group multiple ports together over a single channel, e.g. ports 1 to 4 routed over channel 0, and ports 5 to 8 routed over channel 1.



Relationship between channels and ports on a Router Mk2S in interface mode when receiving

Opening and Closing Device Channels

A channel to a local device can be opened on a device by calling the `OpenChannel()` method and passing in the channel number on the device to open. This function returns a `Channel` object which can be used to identify the channel in future function calls and to transmit and receive packets on. When a channel is no longer required, it can be closed by calling the `CloseChannel()` method on the `Channel` object.

A device channel may be opened uni-directionally or bi-directionally. A channel on a device cannot be opened more than once in the same direction at the same time. For example, one application can open channel 1 of a device as an out channel to transmit packets, while another application can open channel 1 of the same device as an in channel to receive packets. The second application could not open that channel as an out channel, or an in/out channel, until the first application closes its channel. Channel 2 of the device could still be opened, however, in any direction. The direction of the channel is specified when it is opened by passing in one of the following values which are defined as constants in the `Channel` class: `Channel::DIRECTION_IN`, `Channel::DIRECTION_OUT` or `Channel::DIRECTION_INOUT`.

In addition to the channel number and channel direction, a third parameter to the `OpenChannel()` function indicates whether traffic received on the channel should be buffered. If the channel is not opened in buffered mode, packets will be discarded if they are received when no receive operation is waiting for packets. It is strongly recommended that the channel should be buffered for channels which are receiving traffic (in or in/out channels) as traffic may be lost. For channels which are transmitting only (out channels) the buffering status is ignored.

The following code shows the process of opening and closing a bidirectional channel on channel one of a device in buffered mode.

```
Channel channel;
const unsigned char channelNumber = 1;

// open a channel on the device
channel = pDevice->OpenChannel(Channel::DIRECTION_INOUT, channelNumber, 1);

// if channel failed to open
if (!channel.IsOpen())
{
    cout << "Unable to open a channel to the device." << endl;
    return EXIT_FAILURE;
}

// \todo do something with the channel

// close the channel when finished using it
channel.CloseChannel();
```

Transmitting Packets

Packets are transmitted in STAR-System by submitting transfer operations consisting of one or more stream items, representing the packets to be transmitted. Various types of stream items can be transmitted such as whole data packets, parts of a data packet (a data packet chunk), time-codes or errors. To transmit a packet, you must declare an instance of a `Packet` object and call its `CreatePacket()` function. The parameters to this function include the address of the packet (which can be `NULL`), the bytes of the packet, the length of the packet, and the end of packet marker type, which can be EOP (`Packet::EOP_TYPE_EOP`), EEP (`Packet::EOP_TYPE_EEP`) or none (`Packet::EOP_TYPE_NONE`) to send a packet which is not terminated.

Once a packet has been created, a transmit operation must be created to transmit it. Transmit operations can transmit an array of stream items, allowing multiple packets to be transmitted in quick succession. A transmit operation is created by declaring an instance of a `TransmitOperation` object and calling its `CreateTxOperation()` function, passing in the array of packet stream items and the number of elements in the array.

Once created, a transmit operation can be submitted to a previously opened channel using the `SubmitTransferOperation()` function on the `Channel` object. Both transmit and receive operations can be submitted using this function, and the two types of operation are collectively referred to as “transfer” operations. Once an operation has been submitted, its status can be obtained by calling its `GetTransferOperationStatus()` function. To wait on the operation completing its `WaitOnTransferOperationCompletion()` function can be called. The latter expects a parameter which is the time to wait on the operation completing in milliseconds or a value of `TransferOperation::WAIT_INDEFINITELY` to wait indefinitely.

Once a transfer operation is complete, it can be disposed of by calling its `DisposeTransferOperation()` method. This will stop the operation if it is in progress and free up all memory used. Any stream items which have been created can be destroyed separately by calling their Destroy functions, e.g. `DestroyPacket()` for `Packet` objects. Note: the Dispose and Destroy methods will be automatically called if required when any transfer operation or stream item object goes out of scope or is deleted, i.e. in the object destructors. Alternatively, once a transfer operation has completed, it can be resubmitted by calling the `SubmitTransferOperation()` function again on the relevant `Channel`, assuming it has not been disposed of. This can be useful when transmitting the same traffic multiple times or when repeatedly receiving traffic. In both cases, a transfer operation can be created once and the operation resubmitted each time it completes. Stream items can also be reused at any time. For example, a single data packet stream item could be created and provided in the calls to create to two transmit operations. These transmit operations could then be submitted to two different channels at the same time, resulting in the same packet being transmitted on each channel.

The code below shows the process of creating a packet and a corresponding transmit operation, submitting the operation, waiting on it completing, and then disposing of the operation and the packet.

```
Packet packet;
int result;
TransmitOperation transmitOp;
unsigned char transmitBuffer[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
const int transmitBufferLength = sizeof(transmitBuffer);
TransferOperation::TransferStatus status;

// create a 10 byte packet to be transmitted, terminated with an EOP
result = packet.CreatePacket((Address *)NULL,
    transmitBuffer, transmitBufferLength,
    Packet::EOP_TYPE_EOP);

// if failed to create packet
if (result != 1)
{
    cout << "Couldn't create the packet to be transmitted." << endl;
    return EXIT_FAILURE;
}

// create a transmit operation to transmit the packet
result = transmitOp.CreateTxOperation(&packet, 1);
if (result != 1)
{
    cout << "Couldn't create the transmit operation." << endl;
    packet.DestroyPacket();
    return EXIT_FAILURE;
}
```



```

// start transmitting the packet
channel.SubmitTransferOperation(&transmitOp);

// wait indefinitely on the transmit completing
status = transmitOp.WaitOnTransferOperationCompletion(
    TransferOperation::WAIT_INDEFINITELY);

// if transmit didn't complete successfully
if (status != TransferOperation::TRANSFER_STATUS_COMPLETE)
{
    cout << "Could not transmit packet, error of " << status << "." << endl;
}

// dispose of the transmit operation
transmitOp.DisposeTransferOperation();

// destroy the transmit packet
packet.DestroyPacket();

```

Since the above code is likely to be used quite often, STAR-System provides a convenience function on the `Channel` object, `TransmitPacket()`, to simplify these steps. This function isn't as flexible as using the functions from the previous example, because it only allows one packet to be sent at a time, and it may not provide the same performance as using the individual functions shown above. For example, a transmit operation can't be resubmitted when it completes. It does greatly simplify the code required, however, as shown in the following example.

```

unsigned char transmitBuffer[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
const int transmitBufferLength = sizeof(transmitBuffer);
TransferOperation::TransferStatus status;

// transmit a packet, waiting indefinitely on being transmitted
status = channel.TransmitPacket(transmitBuffer, transmitBufferLength,
    Packet::EOP_TYPE_EOP, TransferOperation::WAIT_INDEFINITELY);

// if transmit didn't complete successfully
if (status != TransferOperation::TRANSFER_STATUS_COMPLETE)
{
    cout << "Could not transmit packet, error of " << status << "." << endl;
}

```

This example provides the same functionality as in the earlier transmit example, but with only one function call. The arguments to `Channel.TransmitPacket()` include the buffer to transmit, the length of the buffer, the end of packet marker type to add to the end of the buffer, and the timeout period in milliseconds. The function returns the transfer status of the operation.

Receiving Packets

Packets are received using steps similar to those required to transmit packets: an operation is created and it is then submitted to a channel. A receive operation is created by declaring an instance of a `ReceiveOperation` object and calling its `CreateRxOperation()` function, providing the number of stream items to receive and the types of stream item to receive. Passing `-1` for the number of stream items results in an unlimited number of stream items being received until the operation is cancelled. Note that the unlimited option should be used with caution as it can lead to out of memory errors.

The types of stream item that can be received include whole data packets, data packet chunks and time-codes. A receive operation can receive multiple stream item types, although note that only one of packets or packet chunks can be specified, not both. When receiving time-codes and data packet chunks, the sequence of time-codes and data characters will be maintained, e.g. a time-code may be received in the middle of a packet, which would result in a data packet chunk for the start of the packet, a time-code, and then a data packet chunk for the end of the packet being received. When receiving both time-codes and full packets, a time-code in the middle of a packet will be received by the operation before the packet. This is because the packet cannot be received by the operation until all of its bytes and end of packet marker have been received.

The process of submitting a receive operation, checking its status, waiting on it completing and disposing of it is achieved using the same steps as for transmit operations (see sample code below). When a receive operation completes, the stream items received can be obtained by calling the `GetTransferItem()` and `GetTransferItemList()` functions on the `ReceiveOperation` object, whilst the `GetTransferItemCount()` function returns the number of stream items received by the operation. Once the number of stream items has been obtained, the index of a stream item can be passed to `GetTransferItem()` to obtain a pointer to a new instance of a stream item object which encapsulates the relevant stream item. `GetTransferItemList()` returns the full array of stream items, indicating the number of elements in that array by updating the variable pointed to by the pointer argument passed to the function. NOTE: it is not necessary to call the Destroy function on stream item objects returned by these functions, e.g. `DestroyPacket()`. They are owned by the receive operation, and will continue to exist until the receive operation is disposed of. However, the stream item objects themselves which are returned by `GetTransferItemList()` must be freed when no longer required by calling the C++ delete operator.

The STAR-System C++ API contains classes to represent the different forms of stream item which may be transmitted and received, e.g. `Packet`, `DataChunk`, `TimeCode`. The `GetTransferItem()` function will return a pointer to a parent form of these classes (`StreamItem`) which must be cast to the appropriate child class in order to use the functions specific to the stream item type. For example, to obtain the data for a received packet one can call the `GetPacketData()` function on the `Packet` object, while the end of packet marker type can be obtained by calling `GetPacketEOP()`. Note that a pointer to the data is returned by `GetPacketData()`, and this must be destroyed when no longer required by calling `DestroyPacketData()`. The type of a stream item can be determined if required prior to performing a cast by calling `StreamItem.GetStreamItemType()`.

The following example shows how to create a receive operation to receive one SpaceWire packet and then wait on it completing. Once the operation is complete, the bytes in the received packet are printed.

```
Packet *pRxPacket;
ReceiveOperation receiveOp;
unsigned char *pReceiveBuffer;
TransferOperation::TransferStatus status;
unsigned long receiveBufferLength, i;

// create a receive operation for receiving one packet
if (!receiveOp.CreateRxOperation(1, ReceiveOperation::RECEIVE_PACKETS))
{
    cout << "Couldn't create receive operation, exiting." << endl;
    return EXIT_FAILURE;
}

// start receiving a packet on a previously opened channel
channel.SubmitTransferOperation(&receiveOp);
```

```

// wait indefinitely on a packet being received
status = receiveOp.WaitOnTransferOperationCompletion(
    TransferOperation::WAIT_INDEFINITELY);
if (status != TransferOperation::TRANSFER_STATUS_COMPLETE)
{
    cout << "Could not receive packet, error of " << status << "." << endl;
    receiveOp.DisposeTransferOperation();
    return EXIT_FAILURE;
}

// get the packet stream item received
pRxPacket = (Packet *)receiveOp.GetTransferItem(0);
if (!pRxPacket)
{
    cout << "Unable to get the stream item received." << endl;
    receiveOp.DisposeTransferOperation();
    return EXIT_FAILURE;
}

// get the bytes of the packet received
pReceiveBuffer = pRxPacket->GetPacketData(&receiveBufferLength);
if (!pReceiveBuffer)
{
    cout << "Unable to get the contents of the packet." << endl;
    delete pRxPacket;
    receiveOp.DisposeTransferOperation();
    return EXIT_FAILURE;
}

// display the bytes in the packet
cout << "Received packet contents:";
for (i = 0; i < receiveBufferLength; i++)
{
    cout << " " << hex << (int)pReceiveBuffer[i];
}
cout << endl;

// destroy the packet data obtained, and the received packet itself
pRxPacket->DestroyPacketData(pReceiveBuffer);
delete pRxPacket;

// dispose of the receive operation
receiveOp.DisposeTransferOperation();

```

As with the transmit functions, STAR-System also provides a convenience function on the [Channel](#) object to simplify the steps required to receive a single packet: [ReceivePacket\(\)](#). This function can be used to receive a packet into a previously allocated buffer. As with [TransmitPacket\(\)](#), it is not as flexible or as efficient as using the functions above. For example, the packet length is limited by the size of the buffer, and if the packet received is larger than the buffer, the rest of the packet is dropped. Again, to achieve the highest performance and lowest CPU utilisation, the individual functions described above should be used.

If you simply wish to receive one packet, however, and aren't concerned about performance, the example below shows code to receive a single packet, providing the same functionality as the example above.

```

unsigned char receiveBuffer[10];
TransferOperation::TransferStatus status;
unsigned int receiveBufferLength = sizeof(receiveBuffer), i;

```

```
Packet::EopType eopType;

// receive a packet, waiting indefinitely on it being received
status = channel.ReceivePacket(receiveBuffer, &receiveBufferLength,
                              &eopType, TransferOperation::WAIT_INDEFINITELY);
if (status == TransferOperation::TRANSFER_STATUS_COMPLETE)
{
    // display the bytes in the packet
    cout << "Received packet contents:";
    for (i = 0; i < receiveBufferLength; i++)
    {
        cout << " " << hex << (int)receiveBuffer[i];
    }
    cout << endl;
}
else
{
    cout << "Could not receive packet, error of " << status << "." << endl;
}
```

The arguments to `ReceivePacket()` include the buffer to receive in to, a pointer to a variable which should contain the length of the buffer and which will be updated to contain the length of the packet received, a pointer to a variable which will be updated to contain the end of packet marker type, and the timeout period in milliseconds. The function returns the status of the operation.

Complete Transmit and Receive Example Code

The code below brings all of the information provided above together to show how to transmit and receive a packet. It demonstrates the following functionality:

- Get the list of devices connected to the PC
- Get the type of the first device and the channels on that device
- Open a bidirectional channel to channel 1 of the device
- Create a transmit operation to transmit a single packet and a receive operation to receive a single packet
- Submit the transmit and receive operations on the channel
- Wait on the receive operation completing
- Display the contents of the received packet
- Wait on the transmit operation completing
- Dispose the transmit and receive operations and close the channel to the device

```

#include "star_system.hpp"
#include "device_list.hpp"
#include "device.hpp"
#include "channel.hpp"
#include "packet.hpp"
#include "transmit_operation.hpp"
#include "receive_operation.hpp"
using namespace stardundee::com::starsystem::general;
using namespace stardundee::com::starsystem::channels;
using namespace stardundee::com::starsystem::streamitems;
using namespace stardundee::com::starsystem::transferoperations;

void TransmitAndReceiveExample::DoTransmitAndReceive()
{
    DeviceList deviceList;
    Device *pDeviceToUse;
    U32 deviceCount, i;
    char deviceType[Device::STR_DEVICE_TYPE_LEN];
    size_t stringLen;
    STAR_CHANNEL_MASK channelMask;
    Channel channel;
    Packet transmitPacket, *pReceivedPacket;
    StreamItem *pStreamItem;
    TransmitOperation transmitOp;
    ReceiveOperation receiveOp;
    unsigned char transmitBuffer[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    unsigned char *pReceiveBuffer;
    const int transmitBufferLength = sizeof(transmitBuffer);
    TransferOperation::TransferStatus status;
    unsigned int packetCount, receiveBufferLength;
    const unsigned char channelNumber = 1;

    // get the list of devices present
    deviceCount = deviceList.Populate();

    // if there are no devices present
    if (!deviceCount)
    {
        cout << "No devices present." << endl;
        return;
    }

    // display the number of devices
    cout << deviceCount << " device(s) present." << endl;

    // get a pointer to the first device on the list
    // \note If the device list is freed or re-populated the pointer obtained
    //       will no longer be valid. Use DeviceList.CopyDeviceAt() instead to
    //       obtain a memory-allocated copy of the selected Device on the list
    //       (which must be freed when no longer required using the C++ delete
    //       operator)
    pDeviceToUse = deviceList[0];

    // get the type of the device as a string
    stringLen = pDeviceToUse->GetDeviceTypeAsString(deviceType);
    if (stringLen)
    {
        cout << "Device type = " << deviceType << endl;
    }
}

```

```

else
{
    cout << "Unable to obtain the device type." << endl;
}

// get the channels on the device
channelMask = pDeviceToUse->GetDeviceChannels();

// check that the channel to use exists on the device
if (!(channelMask & (1 << channelNumber)))
{
    cout << "Device doesn't have a channel " << (int)channelNumber << endl;
    return;
}

// open a bidirectional buffered channel to the device
channel = pDeviceToUse->OpenChannel(Channel::DIRECTION_INOUT,
                                    channelNumber, 1);

if (!channel.IsOpen())
{
    cout << "Unable to open a channel to the device." << endl;
    return;
}

// create a packet to be transmitted, terminated with an EOP
if (!transmitPacket.CreatePacket((Address *)NULL, transmitBuffer,
                                transmitBufferLength,
                                Packet::EOP_TYPE_EOP))
{
    cout << "Couldn't create the stream item to be transmitted." << endl;

    // clean up resources created thus far, prior to exiting function
    channel.CloseChannel();

    // \note Some resources (such as transfer operations and packets) will
    //         automatically dispose of and/or destroy themselves if required when
    //         they go out of scope or are deleted, if they haven't already been
    //         'manually' disposed of by the user. Therefore, explicit calls to
    //         Dispose and Destroy methods need not be utilised during clean up
    //         stages and early exit stages if preferred, instead letting the
    //         objects clean up after themselves when the function exits and they go
    //         out of scope.
    //
    //         It is worth paying attention to the order in which objects may be
    //         disposed however. For example, if a packet is destroyed while a
    //         transmit operation is still in the process of attempting to transmit
    //         the packet's data it may cause unexpected side effects.
    //
    //         This example makes explicit use of Dispose and Destroy methods for
    //         the purposes of illustration.

    // exit the function
    return;
}

// create a transmit operation to transmit the packet
if (!transmitOp.CreateTxOperation(&transmitPacket, 1))
{
    cout << "Couldn't create the transmit operation." << endl;

    // clean up resources created thus far, prior to exiting function

```

```

        transmitPacket.DestroyPacket();
        channel.CloseChannel();

        // exit the function
        return;
    }

    // create a receive operation for receiving one packet
    if (!receiveOp.CreateRxOperation(1, ReceiveOperation::RECEIVE_PACKETS))
    {
        cout << "Couldn't create receive operation, exiting." << endl;

        // clean up resources created thus far, prior to exiting function
        transmitOp.DisposeTransferOperation();
        transmitPacket.DestroyPacket();
        channel.CloseChannel();

        // exit the function
        return;
    }

    // start receiving a packet
    channel.SubmitTransferOperation(&receiveOp);

    // start transmitting the packet
    channel.SubmitTransferOperation(&transmitOp);

    // wait indefinitely transmit operation completing
    status = transmitOp.WaitOnTransferOperationCompletion(
        TransferOperation::WAIT_INDEFINITELY);

    // wait indefinitely on a packet being received
    status = receiveOp.WaitOnTransferOperationCompletion(
        TransferOperation::WAIT_INDEFINITELY);
    if (status != TransferOperation::TRANSFER_STATUS_COMPLETE)
    {
        cout << "Could not receive packet, error of " << status << endl;

        // clean up resources created thus far, prior to exiting function
        receiveOp.DisposeTransferOperation();
        transmitOp.DisposeTransferOperation();
        transmitPacket.DestroyPacket();
        channel.CloseChannel();

        // exit the function
        return;
    }

    // check there was one packet received
    // (this is a sanity check, and shouldn't be required)
    packetCount = receiveOp.GetTransferItemCount();
    if (packetCount != 1)
    {
        cout << "Unexpected number of packets received: " << packetCount << endl;

        // clean up resources created thus far, prior to exiting function
        receiveOp.DisposeTransferOperation();
        transmitOp.DisposeTransferOperation();

        transmitPacket.DestroyPacket();
        channel.CloseChannel();
    }

```

```
// exit the function
return;
}

// get the stream item received
pStreamItem = receiveOp.GetTransferItem(0);
if (!pStreamItem)
{
    cout << "Unable to get the stream item received." << endl;

    // clean up resources created thus far, prior to exiting function
    receiveOp.DisposeTransferOperation();
    transmitOp.DisposeTransferOperation();
    transmitPacket.DestroyPacket();
    channel.CloseChannel();

    // exit the function
    return;
}

// check the stream item is a SpaceWire packet
// (this is a sanity check, and shouldn't be required)
if (pStreamItem->GetStreamItemType() != StreamItem::STREAM_ITEM_TYPE_PACKET)
{
    cout << "The stream item received is not a packet." << endl;

    // clean up resources created thus far, prior to exiting function
    delete pStreamItem;
    receiveOp.DisposeTransferOperation();
    transmitOp.DisposeTransferOperation();
    transmitPacket.DestroyPacket();
    channel.CloseChannel();

    // exit the function
    return;
}

// cast stream item to packet
pReceivedPacket = (Packet *)pStreamItem;

// get the bytes of the packet received
pReceiveBuffer = pReceivedPacket->GetPacketData(&receiveBufferLength);
if (!pReceiveBuffer)
{
    cout << "Unable to get the contents of the packet." << endl;

    // clean up resources created thus far, prior to exiting function
    delete pStreamItem;
    receiveOp.DisposeTransferOperation();
    transmitOp.DisposeTransferOperation();
    transmitPacket.DestroyPacket();
    channel.CloseChannel();

    // exit the function
    return;
}
```



```

// display the bytes in the packet
cout << "Received packet contents:";
for (i = 0; i < receiveBufferLength; i++)
{
    cout << " " << hex << (int)pReceiveBuffer[i];
}
cout << endl;

// destroy the buffer containing the received packet
// \note The buffer obtained from Packet.GetPacketData() MUST always be
//       freed when no longer required by using Packet.DestroyPacketData(),
//       if it is a valid buffer, i.e. non-NULL. The Packet object will not
//       automatically free the buffer obtained in its destructor, e.g.
//       when it goes out of scope or is deleted. Calls to
//       Packet.DestroyPacket() will not free the buffer either.
pReceivedPacket->DestroyPacketData(pReceiveBuffer);

// free the stream item retrieved from the receive operation
delete pStreamItem;

// dispose of the receive operation and transmit operation
// \note Transfer operation objects (such as ReceiveOperation
//       and TransmitOperation) will automatically dispose of any transfer
//       operation they hold if required when they go out of scope or are
//       deleted. Therefore, the following lines of code may be omitted if
//       preferred.
receiveOp.DisposeTransferOperation();
transmitOp.DisposeTransferOperation();

// destroy the transmit packet
// \note The Packet object will automatically destroy the packet it contains
//       if required when it goes out of scope or is deleted. Therefore,
//       the following line may be omitted if preferred.
transmitPacket.DestroyPacket();

// close the channel to the device
// \note The Channel object does NOT automatically close the channel it refers
//       to when it goes out of scope or is deleted. Therefore, it must be
//       closed by the user when no longer required as below:
channel.CloseChannel();
}

```

When the above code is run on a PC with a loopback cable connected to port 1 of a PCI Mk2, the following output should be displayed:

```

1 device(s) present.
Device type = SpaceWire PCI Mk2.
Received packet contents: 01 02 03 04 05 06 07 08 09 0a.

```

